

ESP32-P4 上的 openvela AI 设备诊断助手

让模型读取真实设备状态，让一次问答可靠回到命令行。

从“能启动”走向“可诊断”

USB 控制台 · 有线联网 · 板端工具
openvela Agent · 云端模型 · NSH 返回

本项目贡献：板级集成、运行时修正、
设备工具与可复用调试方法。

截图位 01 / 待补实机画面

开发板实物与连接

补图：ESP32-P4、USB 与网线。
不用放 API Key 或配置截图。

相同芯片，交付深度可以不同

面向开发者的无屏设备诊断，不是通用聊天终端。

基础：平台能运行

启动 NSH
USB Console
以太网与文件系统

进阶：Agent 能执行

查询真实堆与运行时间
读取 eth0 状态
通过 JSON 返回给模型

重点：过程可复现

异常寄存器与堆链表证据
同步 ask 返回 NSH
构建、配置与验收闭环

差异化来自“真实工具 + 运行可靠性 + 调试证据”，不是重复声明移植成功。

保留上游归属，明确本项目增量

基于开源平台做工程集成与针对性改进，不将上游驱动或 Agent 引擎算作原创。

复用的基础

NuttX / Espressif 的 ESP32-P4 基础
openvela ai_agent 与 ReAct 工具循环
cJSON、mbedTLS、NTP 等组件
已有板卡适配与社区工作

本项目实现 / 修正

USB + EMAC + 存储的目标配置
device_info / network_status 工具
CLI 同步 ask 与资源生命周期
栈 / 堆 / USB / TLS / 时间适配
实机定位与验收流程

设备端执行，云端推理

模型通过工具获取观测值，而不是仅根据提示词猜测设备状态。



两项工具，把语言连接到设备事实

不依赖额外传感器，以板卡已有的系统资源和以太网作为可验证输入。

device_info

内核 / 架构 / 运行时间
堆总量、已用、空闲、最大空闲块
结构化 JSON，便于核对和解释

network_status

eth0 注册、UP、RUNNING 状态
IPv4、网关、掩码、MAC
缺失字段返回 null，而非虚构数据

能力边界：接口“已配置”不代表外网可达；本工具不测 DNS、丢包或链路速率。
演示时用 ifconfig / free 对照，证明回答有设备数据依据。

现场演示：一次问答完成一次诊断

建议提问：请调用 `device_info` 和 `network_status`，概括设备内存与网络配置。

截图位 02 / 待补实机画面

工具问答与 NSH 返回

补图：提问、设备状态回答、`nsh>`。
同时保留工具调用证据；严禁用模拟输出冒充。

三个观察点

- 01 数据能与命令核对
- 02 回答后回到 NSH
- 03 可继续执行下一条命令

不是泛泛“死机”：定位实际任务栈

配置名相似并不代表作用于同一个任务；必须回到真实执行路径。

现象 → 证据 → 修正

set_illm 卡住；getumask 访问异常
INIT 栈实际 2 KB，不是预期 32 KB
调整真实启动栈，启用栈着色验证

实机 ps：NSH 栈峰值 3572 B
超过旧 2048 B 配置。

截图位 03 / 待补实机画面

GDB 异常 + ps 栈证据

补图：mepc / mtval 与修正后的 ps。
可分为上下两张，保留关键数值。

free 无限打印，根因在堆布局

不在显示层截断输出，而是修正内存分配器初始化配置。

链表自环 → 重叠堆 → 共享分配器

GDB: `g_procfs_meminfo->next` 指向自己
检查 `heap / kheap` 返回的 SRAM 区域
关闭独立内核堆，完整重编译

结果：实机反馈 `free` 恢复。
避免以“断开链表”掩盖内存损坏。

截图位 04 / 待补实机画面

自环证据 + 正常 free

补图：`next == self` 为 1 的 GDB 记录，
以及修正后只输出一次并返回 NSH。

跨层适配，让云端请求真正跑通

解决的不是单一 API，而是输入、存储、网络、安全传输和时间的一组约束。

USB 与文件系统

USB RX/TX 环形队列 64 → 2048 B
统一正常日志与控制台输出路径
SmartFS 短名称兼容与写入错误检查

TLS 与时间

启用 ECDHE，清理重建依赖归档
NTP 校时，不再强设固定日期
CLOCK_MONOTONIC 统计请求耗时

已有实机握手：TLSv1.2 / TLS-ECDHE-RSA-WITH-AES-256-GCM-SHA384
边界：证书校验仍为 VERIFY_OPTIONAL，尚未完成生产安全配置。

一次 ask，一次可结束的工作单元

同步执行、复用核心状态、释放请求缓冲区；避免把命令行变成假交互界面。

01

进入

NSH 发起 ask
首次完成核心初始化

02

执行

复用 ReAct 循环
模型请求与工具执行

03

返回

释放请求缓冲
输出一次并返回 NSH

取舍：此模式不提供常驻提醒与 WebSocket 服务；验证重点是连续请求和资源稳定性。

AI 协作不是替代验证，而是加快验证

需求拆解 → 代码建议 → 构建 → 实机反馈 → GDB 证据 → 定点修正。

已发生的协作工作

跨仓配置与代码核对
分析串口日志、异常寄存器和链表
将修正编译后交由板端验证

待归集的开发 Skill

版本与 ELF 一致性检查
栈 / 堆 / USB / TLS 排障路径
每一步的验收条件与失败分支

工作量快照：3 个仓库、30 个已跟踪文件存在未提交改动，另有新配置与工具文件。
这是当前工程覆盖范围，不是原创代码量，也不包括全部历史工作。

用可核对的证据结束演示

展示已验证结果；把发布前补测项明确留在清单中。

截图位 05 / 待补实机画面

网络 / 时间 / 连续请求

补图：ifconfig、date、两次 ask、free。
可用已有 ping 记录作辅证，不宣称长期零丢包。

已有实机反馈

DHCP、局域网 / 公网 ping
TLS 与问答、NSH 返回
free、日期恢复正常

最终补测

新文件名、配置对齐、重启持久化
连续请求资源变化、工具调用证据

交付的不只是启动画面

把一个能跑的板卡，推进为能读取状态、能解释结果、能继续操作的 AI 终端。

真实

板端 API → JSON
模型基于观测值解释

可控

ask 结束返回 NSH
不依赖额外外设

可追溯

故障证据对应修正
代码与测试边界清楚

项目仓库: [zealsoftstudio / contest2026_124_shenyiyangdelaoshi](#)

附录：核心改动如何定位

以下为代码导航。最终提交需要补齐基线、跨仓 PR 与正式发布记录。

目标配置 / 启动	nuttx: configs/usbnet/defconfig、esp32p4_bringup.c
USB / 存储	nuttx: esp_usbserial.c、esp_board_spiflash.c
DHCP 初始化	apps: netutils/netinit/Kconfig、netinit.c
同步 Agent	ai_agent: agent_main.c、core/agent_loop.c
设备工具	ai_agent: tools/tool_esp32p4.c、tool_registry.c
时间 / 文件名	ai_agent: core/agent_trace.c、tools/skill_loader.c

附录：发布前须如实说明的限制

把限制说清楚，比用未经验证的功能充数更有说服力。

- 01 依赖云端模型和网络；天气、搜索密钥未配置。
- 02 CLI ask 不运行后台 cron / heartbeat / WebSocket。
- 03 TLS VERIFY_OPTIONAL 尚未完成生产级证书验证。
- 04 PSRAM 未作为用户堆；不宣称已支持所有硬件外设。
- 05 SmartFS：当前 .config 为 16，defconfig 为 48，发布前需对齐复验。
- 06 新短文件名、连续请求与干净构建仍需最终验收。